

Take-Home Assignment — Live Ops Board

Senior / Staff Frontend Engineer · TransVirtual · Candidate brief

Overview

Build a live operations board: a high-volume shipment table that stays smooth and responsive while statuses update in real time. We care most that it runs end-to-end as a workable demo — including the live feed, which you build yourself — and that your architecture write-up explains the decisions that make it fast.

What we provide

- `shipments_10k.csv` — 10,000 shipments with columns: `reference`, `customer_name`, `status`, `last_update`.

That is all. How you turn a static CSV into a live-updating system — the feed, the transport, the tooling — is yours to design, and it is part of what we evaluate.

Requirements

1. Mock live feed: build a mechanism that loads `shipments_10k.csv` as the initial snapshot and then pushes randomized status updates — a few hundred per second at peak, rate configurable. WebSocket server, in-process generator, worker: your call; be ready to defend it.
2. Table (React + TypeScript): a virtualized list rendering the 10k shipments with `reference`, `customer`, `status`, and `last-update` time. Scrolling stays smooth while updates stream in.
3. Live updates: rows update in place with a brief visual highlight on change. The UI must remain responsive at your feed's peak rate — we will check this in the demo.
4. Controls: filter by status and a text search over `reference/customer`, both staying correct and responsive while data churns underneath.
5. `ARCHITECTURE.md` (about a page): feed design, state-management choice, data layer, render-performance budget and how you met it, plus a short written note on how you would add offline tolerance (design only, no implementation).
6. Tests for the genuinely tricky parts — e.g., update batching or filtering under churn. No need for broad UI snapshot coverage.
7. Workable demo: your app and your feed start with clearly documented commands (a single command is ideal), with no external services required.

Ground rules

- Timebox: 4–6 hours of effort, due within 2 days of receiving this brief (a 3rd day is available on request). Stop at the timebox and write down what you would do next — unfinished-but-well-reasoned beats gold-plated.
- AI tools (Claude Code, Copilot, Cursor, etc.) are allowed and encouraged — we work AI-first. Include a short AI-usage note: which tools, what was generated vs. hand-written, and at least one thing the AI got wrong and how you caught it.
- Tooling is your call — languages are fixed by the requirements, but frameworks, libraries, project layout, and how you run things locally are up to you. Choose deliberately; we will ask why. Skip authentication, deployment, and anything not listed in the requirements.
- You will demo this live (about 5 minutes) at the start of your final interview and extend it during the session — build something you can talk about.

What to submit

- A Git repository link (or zip) with the full source.
- `README.md`: how to run the demo (the exact commands), key decisions — including your tooling choices and why — and your “what I’d do next” list.
- Your AI-usage note (in the `README` or separate).
- `ARCHITECTURE.md` as described above.

How we evaluate

- The demo is the deliverable: if we cannot run your project with the documented commands, we will ask you once to fix the setup; if it still doesn't run, we can't progress the application. The demo matters more than any individual feature.
- We look at: how the demo holds up under your feed's load · feed and system design · setup and tooling choices · architecture and state management · code clarity · performance engineering · how well the write-up communicates trade-offs · judgment in how you used AI tools.
- In the final interview you'll demo the board, defend the architecture as we scale the constraints (10× rows, multiple boards, offline depots), and discuss how you'd grow a young team around these standards.